

Tasks — Full Working Flow, Gaps & Suggestions

Companion to *Tasks Module — Documentation*. End-to-end runtime flow, then what is missing / unsafe and recommended next steps.

EHRMS · app_backend + hrms (Flutter) · Generated 2026-06-26 · reflects code as currently read

Part A — Full working flow

A.1 Actors & surfaces

Actor	Surface	Does
Field staff	Flutter app (<code>screens/geo/</code>)	Receives task, travels, proves arrival, completes
Admin / scheduler	Web (external) + same APIs	Creates/assigns, reopens, watches live map, approves
Customer	Email	Receives OTP, shares it with staff to confirm the visit
Backend	<code>app_backend</code>	Persists, merges, validates transitions, broadcasts over Socket.io

A.2 End-to-end sequence

STAFF APP		BACKEND	STORES / EXTERNAL
create task	—POST /tasks→	generate TASK-XXXX upsert details businessId from staff	tasks (minimal) task_details
list / open	—GET /tasks/:id→	merge task+details +settings, +battery, +derived travel metrics	← tasks + task_details + task-settings + trackings
START RIDE	—PATCH /tasks/:id {status:in_progress, startLat/Lng}	set rideStartedAt/Location	
every ~15s	—POST /tracking/store {lat,lng,battery,movementType}	outlier-filtered (client)	trackings (+offline queue)
(live view)	—POST /tasks/:id/location→	emit Socket.io	rooms: task:, admin:*
(optional)	—POST /tracking/exit {exitType: hold exited}	push exit[], set status	tasks.task_exit + details.exit
resume	—POST /tracking/restart→	status in_progress, push restarted[]	
ARRIVED	—POST /tracking/arrived {lat,lng,trip...}	status=arrived flag overridencustomerlocation (>50m from customer) flag overridendestinationlocation (>50m / changed)	details.arrivallocation
PROVE VISIT	—POST /tasks/:id/photo→	upload to Spaces	details.photoProofUrl, step
	—POST /tasks/:id/selfie→	checkin/checkout	details.arrivedSelfie*
	—POST /forms/responses→	form filled	FormResponse, step
	—POST /tasks/:id/send-otp→	email 4-digit OTP	→ SendPulse → customer
	—POST /tasks/:id/verify-otp→	match otpCode	step.otpVerified
COMPLETE	—POST /tasks/:id/end→	arrived→completed OR →waiting_for_approval (if requireApprovalOnComplete) persist final travel metrics	
REVIEW	—GET /tasks/:id/completion-report→	task + timeline + route + forms	
ADMIN	—PATCH /tasks/:id {status:reopened}→	reopen completed/rejected task	

A.3 The split/merge cycle (every write)

Write: controller keeps `tasks` minimal → `$unset` extended keys (`buildUnsetExtended`) and upsert everything into `task_details` (`upsertTaskDetails`).

Read: `mergeTaskWithDetails` rejoins → `mergeTaskSettings` overlays company flags → travel metrics recomputed from `trackings` → battery annotated (±60 min) → status normalized. App sees one flat object.

A.4 Status & resume rules

- `hold` / `holdOnArrival` → staff resumes themselves via `/tracking/restart`.
- `exited` / `exitOnArrival` → only after an admin sets `reopened`, then staff resumes.

- Arrival never blocks on distance — it only *flags* `overridencustomerlocation` when > 50 m from the customer's stored GPS.
- `endTask` is valid only from `arrived` / `holdOnArrival` / `reopenedOnArrival`, and is idempotent.

A.5 Task settings that are used

Company behaviour is controlled by the `task-settings` collection (`TaskSettings`). Settings are looked up per company (by `companyId` or `businessId`) and overlaid onto every task at read time by `mergeTaskSettings` . There are exactly four settings:

Setting	Default	What it controls	Status in code
<code>enableOtpVerification</code>	false	When on, the task is treated as "OTP required": the staff member must email a 4-digit code to the customer and verify it. Surfaced to the app as <code>customFields.otpRequired</code> .	Used
<code>requireApprovalOnComplete</code>	false	When on, finishing a task does not mark it <code>completed</code> — it moves to <code>waiting_for_approval</code> for a manager to sign off. Read by <code>endTask</code> .	Used
<code>autoApprove</code>	false	Intended to auto-approve newly created tasks. It is passed to the app, but <code>createTask</code> always uses the request's status (default <code>assigned</code>) and never auto-approves on the backend.	Surfaced only — verify if applied anywhere
<code>staffWhoCanSchedule</code>	empty	Intended whitelist of staff allowed to create/assign tasks. Stored but never checked — <code>createTask</code> lets any authenticated staff create a task.	Not enforced

Note the "required" toggles for **photo**, **form**, and **geofence** do *not* exist in `task-settings` — only OTP does. That is why the app's `isPhotoRequired` / `isFormRequired` / `isGeoFenceRequired` flags are always off (see Gap #1).

Part B — What needs to be implemented (gaps)

These are concrete gaps found in the current code, ordered by impact.

#	Gap	Severity	Detail
1	Required-step flags are dead	High	App gates UI on <code>isPhotoRequired</code> , <code>isFormRequired</code> , <code>isGeoFenceRequired</code> , but the backend never sets <code>customFields.photoRequired</code> / <code>formRequired</code> / <code>geoFenceRequired</code> — only <code>otpRequired</code> is derived. <code>TaskSettings</code> has no toggles for them either. They are effectively always <code>false</code> .
2	Multi-tenant data leak	High	<code>GET /tasks</code> , <code>GET /tasks/:id</code> , <code>GET /tasks/:id/completion-report</code> have no <code>protect</code> and no <code>businessId</code> scoping — any caller can read every company's tasks, customer names, photos, OTP-verified addresses.
3	Completion not gated on required proof	High	<code>endTask</code> checks only status, not whether required photo/OTP/form steps were actually completed. A task marked "OTP required" can be completed with no OTP verified.
4	OTP has no expiry / rate limit	Med	Email says "valid 10 minutes" but <code>verifyOtp</code> never checks <code>otpSentAt</code> . No attempt cap → OTP is brute-forceable (10^4 space) and never expires.
5	Geofence is flagged, not enforced	Med	Arrival > 50 m only sets an override flag. There is no setting to <i>require</i> arrival within a radius, and nothing blocks a far-away arrival/complete.
6	<code>staffWhoCanSchedule</code> not enforced	Med	Setting exists but <code>createTask</code> never checks it — any authenticated staff can create/assign tasks.
7	<code>autoApprove</code> likely unused on backend	Low	Surfaced to the app but <code>createTask</code> always uses body status (default <code>assigned</code>); confirm whether auto-approve is applied anywhere or is dead.
8	No cancel / soft-delete route	Low	<code>cancelled</code> exists in the status map but there is no endpoint to cancel a task or remove a wrongly-created one.
9	Photo/selfie size & type only checked locally	Low	multer enforces 5 MB + JPG/PNG, but there is no server-side image dimension / EXIF / content check before pushing to Spaces.
10	No automated tests for the task lifecycle	Low	Transition matrix, merge/split, and travel-metric recompute are intricate and currently unguarded by tests.

Part C — Suggestions

C.1 Close the required-step loop (fixes #1, #3, #5)

1. Add to `TaskSettings.settings` : `enablePhotoProof` , `enableForm` , `enableGeoFence` (+ `geoFenceRadiusM` , default 50).
2. In `mergeTaskSettings` , surface them as `customFields.photoRequired` / `formRequired` / `geoFenceRequired` next to the existing `otpRequired` .
3. In `endTask` , before completing, assert each *required* step is done (`progressSteps.photoProof` , `otpVerified` , `formFilled` ; arrival within radius when geofence required) → else `400` with the missing list.

C.2 Lock down reads (fixes #2)

- Add `protect` to `GET /tasks` , `/tasks/:id` , `/tasks/:id/completion-report` .

- Scope every query by the caller's `businessId` (and for staff, restrict to `assignedTo = self` unless admin). Drop the unscoped "fetch all settings" fallback in `mergeTaskSettings`.

C.3 Harden OTP (fixes #4)

- Reject in `verifyOtp` when `now - otpSentAt > 10 min`.
- Track `otpAttempts`; lock after 5 wrong tries and require re-send.
- Clear `otpCode` on success so it can't be replayed.

C.4 Operational polish

- **Scheduler permission:** enforce `staffWhoCanSchedule` in `createTask` (empty list = anyone, to stay backward-compatible).
- **Cancel route:** `PATCH /tasks/:id` → `cancelled` with reason; exclude cancelled from active lists/counts.
- **Indexes:** ensure indexes on `tasks.assignedTo`, `tasks.expectedCompletionDate`, `task_details.taskId`, and `trackings.taskId + timestamp` — the paginated list and travel-metric recompute scan these heavily.
- **Single source for status spellings:** the dual-maintained `STATUS_GROUP_SPELLINGS` (backend) and `Task.statusFromJson` (app) drift easily — generate one from the other, or add a contract test.
- **Tests:** add lifecycle tests for the transition matrix, merge/split round-trip, and travel-metric recompute (these are the highest-risk, most intricate parts).
- **Idempotency on writes:** `storeTracking` already retries from an offline queue — add a client-supplied dedupe key so retries can't double-insert tracking points.

C.5 Priority order

Order	Item	Why first
1	Lock down reads (C.2)	Active data-exposure risk across tenants
2	Required-step enforcement (C.1)	Core promise of the feature is currently unenforced
3	OTP hardening (C.3)	Small change, removes a real abuse vector
4	Permissions, cancel, indexes, tests (C.4)	Correctness & scale hardening

Note: gaps are from a static read of the code. Before implementing, confirm whether the external web backend already covers any of these (e.g. auto-approve, scheduler permission) so work isn't duplicated.